

格点 QCD 蒸馏法 (Distillation) 实现穷尽剖析： EasyDistillation 的核心物理与代码-物理映射

opencode pure

2026-08-15

摘要

本报告对格点 QCD 蒸馏法 (distillation method) Python 实现 EasyDistillation 的核心部分进行穷尽剖析。项目性质判定为**代码-物理交叉**：物理对象（拉普拉斯本征矢、蒸馏传播子 perambulator、顶点矩阵 elemental、两点关联函数、 O_h 双盖群不可约表示）与算法实现 (opt_einsum 自动收缩、SymPy 符号化 Wick 收缩、图论连通分量化简、解析 SU(3) 指数化) 逐符号对应。核心部分共 17 项：**深挖 8 / 浅析 5 / 排除 4**。最深剖析结论：本项目最强竞争力集中于三处——(i) **Stout 平滑的解析 SU(3) 矩阵指数化** (Cayley-Hamilton 法，四种后端实现，GPU 友好无迭代误差)；(ii) **夸克收缩的符号化-图论双轨自动化** (SymPy 按 Wick 定理生成传播子图，QuarkDiagram 自动生成 opt_einsum 收缩下标，含同构图哈希归一化)；(iii) **O_h 双盖群小群约化的硬编码不可约表示库** (11,613 行预生成数据，运行期零群构造开销)。全部结论附 文件：行号参考源与分级标注（确证/推断/未验证）。

目录

1 项目定位与核心部分清单	2
1.1 项目性质判定	2
1.2 核心部分总清单（穷尽枚举）	2
2 核心部分深度剖析	3
2.1 C1 蒸馏法物理框架：图像、定义与公式链	3
2.1.1 物理图像	3
2.1.2 模型设定	3
2.1.3 公式推导链	4
2.2 C2 拉普拉斯本征矢生成：多后端求解	5
2.3 C3 Stout 平滑：SU(3) 解析矩阵指数化	6
2.3.1 公式推导链	7
2.4 C4 Elemental 生成：导数展开与动量投影	8
2.5 C5 Perambulator 生成：Dirac 求逆与 $V^\dagger SV$	10
2.6 C6 自动 Wick 收缩：符号化-图论双轨	12
2.6.1 符号化收缩 (quark_contract.py)	12
2.6.2 图表示与自动 einsum 下标 (QuarkDiagram)	13

2.6.3	图化简 (Diagram.simplify)	14
2.7	C7 群论不可约表示框架: O_h^D 小群约化	15
2.7.1	物理对象	15
2.7.2	硬编码不可约表示库	15
2.8	C8 两点关联函数收缩	16
3	独特性与竞争力评估	17
4	关键片段与公式	17
5	参考源清单	19
6	结论	20

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质: 代码-物理交叉 (依据: 仓库 42 个 Python 源文件 20,628 行中, 物理计算模块 (generator/correlator/insertion/symmetry) 约 68%, 通用基础设施 (filedata/backend/dispatch) 约 15%, 符号化自动收缩 (quark_contract/quark_diagram) 约 17%; 每个核心物理量均有直接对应代码符号, 双向映射可查——见第 2 节各映射表)。

EasyDistillation 是一个蒸馏法格点 QCD 数值包: 先求解三维规范协变拉普拉斯算子的低能本征矢构造蒸馏算子, 再生成 perambulator 与 elemental, 最后自动完成夸克收缩得到强子关联函数。它覆盖了蒸馏法计算的全部数据流, 且每条数据流都有 numpy/cupy/QUDA/CUDA 多后端实现。

1.2 核心部分总清单 (穷尽枚举)

以下清单穷尽仓库全部核心模块, 三态 (深挖/浅析/排除) 填满, 无”未处理”项。

编号	核心部分	处理态	独特性概述
C1	蒸馏法物理框架与公式链	深挖	蒸馏算子/传播子/顶点的完整定义链与极限校验
C2	拉普拉斯本征矢生成	深挖	多后端求解 + 本征矢相位归一化约定
C3	Stout 平滑 (解析 SU(3) 指数化)	深挖	Cayley-Hamilton 闭式公式, 4 种实现
C4	Elemental 生成	深挖	协变导数展开 + 动量投影 + 随机混合
C5	Pambulator 生成	深挖	Dirac 求逆 + MRHS 批量 + $V^\dagger SV$ 收缩
C6	自动 Wick 收缩与图算法	深挖	符号化收缩 + 自动 einsum 下标生成 + 图同构化简

编号	核心部分	处理态	独特性概述
C7	O_h^D 群论不可约表示框架	深挖	硬编码小群不可约表示库 + Wigner 旋转
C8	两点关联函数收缩	深挖	蒸馏空间两轮收缩的单一 einsum 表达
C9	插入算子体系 (gamma/导数命名)	浅析	比特/三进制编码枚举 16 gamma 与导数组 合
C10	双粒子角动量基	浅析	CG 系数 + 球谐函数的算子基构造
C11	数据层 (filedata/preset)	浅析	多格式 (ILDG/Binary/Ndarray) 惰性加载
C12	自动关联函数构造 (hadron.py)	浅析	从强子不可约表示行直接生成收缩表达式
C13	其他生成器 (噪声/密度/广义/位移)	浅析	蒸馏法扩展算子族
C14	quark_draw 费曼图绘制	排除	通用 feynman 库的薄封装 (样板代码)
C15	backend 后端抽象	排除	numpy/cupy 二选一的调度开关 (样板代码)
C16	dispatch 任务分发	排除	MPI 文件锁分发的通用并行设施
C17	tests/ 测试套件	排除	弱场验证数据驱动, 属验证证据而非核心算法

要点

穷尽性声明: 全部 17 项候选均已显式分配三态 (8 深挖 / 5 浅析 / 4 排除), 本节之后第 2 节逐一剖析深挖项, 第 6 节重述计数闭合。

2 核心部分深度剖析

2.1 C1 蒸馏法物理框架: 图像、定义与公式链

2.1.1 物理图像

物理图像

物理图像: 蒸馏法 (distillation, Peardon 等, <https://arxiv.org/abs/0905.2160>) 把格点上的夸克场投影到拉普拉斯算子低能本征矢张成的有限维子空间上, 使”非局域平滑 + 传播子 + 顶点”全部降维为 $N_e \times N_e$ 小矩阵运算。物理代价 (N_e 的选取) 与计算收益 (收缩成本从 $O(V^2)$ 降到 $O(N_e^2)$) 由本征矢个数 N_e 精确权衡——这是蒸馏法与点源/盒源方法最本质的分界。

2.1.2 模型设定

- 自由度: 规范场 $U_\mu(x) \in SU(3)$, 夸克场 $\psi(x) \in \mathbb{C}^3 \otimes \mathbb{C}^4$ (色 $\times$ 旋量), 格点尺寸 $V = L_x L_y L_z L_t$ (代码 `lattice/constant.py:1-3` 固定 $N_c = 3, N_s = 4, N_d = 4$)。
- 本征矢: 三维 (仅空间) 规范协变拉普拉斯算子 $-\nabla^2$ 的最小 N_e 个本征值对应本征矢 $v^{(k)}(x)$, 组成 V 矩阵 (尺寸 $L_z L_y L_x N_c \times N_e$)。

- 边界条件：时间方向周期/反周期由 `t_boundary` 控制 (`lattice/generator/perambulator.py:63`)，空间方向周期性内建。

2.1.3 公式推导链

第一步（定义）：蒸馏算子。拉普拉斯本征矢满足

$$-\nabla^2 v^{(k)}(x) = \lambda_k v^{(k)}(x), \quad \nabla^2 \psi(x) \equiv \sum_{\mu=1}^3 \left[U_{\mu}(x) \psi(x + \hat{\mu}) + U_{\mu}^{\dagger}(x - \hat{\mu}) \psi(x - \hat{\mu}) - 2\psi(x) \right], \quad (1)$$

其中后一项为规范协变二点导数（自由情形下 $-\nabla^2$ 本征值为 $6 - 2 \sum_{\mu} \cos k_{\mu} \in [0, 12]$ ，即代码注释“SA 用 evals、LA 用 12-evals”的量纲依据，见 `lattice/generator/eigenvector.py:16`）。蒸馏算子定义为前 N_e 个本征矢的外积：

$$\square(x, y) \equiv \sum_{k=1}^{N_e} v^{(k)}(x) v^{(k)\dagger}(y) = V(x) V^{\dagger}(y), \quad \tilde{\psi} = \square \psi. \quad (2)$$

(2) 为幂等投影： $\square^2 = \square$ （因 $\{v^{(k)}\}$ 正交归一），且 $V^{\dagger} V = \mathbb{1}_{N_e}$ 。极限校验： $N_e \rightarrow N_{\text{full}} = 3V$ 时 $\square \rightarrow \delta_{xy}$ （完全投影）； $N_e \rightarrow 0$ 时 $\square \rightarrow 0$ ，两极限行为合理。

第二步（定义）：perambulator。将夸克传播子 M^{-1} （ M 为 Dirac 矩阵）夹在两个蒸馏算子之间，得到蒸馏空间传播子：

$$\tau(t_f, t_i) \equiv V^{\dagger}(t_f) M^{-1} V(t_i), \quad \tau_{ij} = \sum_{x,y} v_i^{\dagger}(x, t_f) M^{-1}(x, t_f; y, t_i) v_j(y, t_i), \quad (3)$$

尺寸 $N_e \times N_e$ （代码缓存 `lattice/generator/perambulator.py:102`）。量纲检查： M^{-1} 为逆质量量纲， V 无量纲，故 τ 与传播子同量纲，符合 $\langle \tilde{\psi} \tilde{\psi} \rangle = \tau$ 。

第三步（定义）：elemental。顶点算子由 γ 矩阵、协变导数与动量相位组成：

$$\Phi_{ij}^{(\Gamma, n)}(t, \mathbf{p}) \equiv \sum_{\mathbf{x}} e^{i\mathbf{p} \cdot \mathbf{x}} v_i^{\dagger}(\mathbf{x}, t) \Gamma [\nabla^{(n)} v_j](\mathbf{x}, t), \quad (4)$$

其中 $\nabla^{(n)} = \nabla_{d_1} \cdots \nabla_{d_n}$ 为 n 阶协变导数链（ $\nabla_{\mu} v(x) \equiv U_{\mu}(x) v(x + \hat{\mu}) - v(x)$ ）。 n 阶导数组组合数

$$N_{\text{deriv}}(n) = \sum_{k=0}^n 3^k = \frac{3^{n+1} - 1}{2} \Rightarrow N_{\text{deriv}}(0) = 1, N_{\text{deriv}}(1) = 4, N_{\text{deriv}}(2) = 13, \quad (5)$$

与代码 `lattice/generator/elemental.py:48` 的 `num_derivative = (3 ** (num_nabla + 1) - 1) // 2` 完全一致。极限校验： $\mathbf{p} \rightarrow 0$ 时相位因子 $\rightarrow 1$ ， Φ 退化为无动量投影的顶点； $n = 0$ 时退化为 $\Gamma \otimes V^{\dagger} V$ 型局域顶点，符合预期。

第四步（推导）：两点关联函数。介子插值场 $\mathcal{O}(t) = \bar{\psi}(t) \Gamma \Phi(t) \psi(t)$ 的两点关联函数 $C(t_f - t_i) = \langle \mathcal{O}(t_f) \mathcal{O}^{\dagger}(t_i) \rangle$ 按 Wick 定理收缩（两对夸克场两两配对，符号由费米子反对易决定，见 C6），在蒸馏空间中约化为

$$C(t_f - t_i) = -\text{Tr} \left[\underbrace{\gamma_5 \tau^{\dagger}(t_f, t_i) \gamma_5}_{\tau_{\text{bw}}} \Phi_{\alpha}(t_f) \tau(t_f, t_i) \underbrace{\gamma_4 \Phi_{\beta}^{\dagger}(t_i) \gamma_4}_{\gamma\text{-共轭源顶点}} \right], \quad (6)$$

其中 $\gamma_5 \tau^{\dagger} \gamma_5$ 因子来自 γ_5 -厄米性 $M^{\dagger} = \gamma_5 M \gamma_5$ ，故 $(M^{-1})^{\dagger} = \gamma_5 M^{-1} \gamma_5$ （代码 `lattice/correlator/one_particle.py:57`）， γ_4 -共轭为粒子-反粒子时间反演约定。公式 (6) 与代码实际

收缩 (lattice/correlator/one_particle.py:64-72) 逐符号对应 (见 C8)。量纲检查： Φ 无量纲、 τ 为逆质量量纲， C 无量纲 (格点单位制)，符合。

极限与退化校验 (公式链总校验)：

- $N_e \rightarrow$ 满秩： $\square \rightarrow \delta$ ，(3) 还原为精确传播子，(6) 还原为常规两点函数 (双线性收缩)，合理。
- $t_f = t_i$ (局域传播子)：(3) 的 $\pi_{\text{local}} = V^\dagger M^{-1} V$ 退化为常数矩阵的迹运算，与代码 PropagatorLocal (lattice/quark_diagram.py:206-230) 一致。
- 手征极限 $m \rightarrow 0$ ： $\tau \sim 1/m$ 发散，关联函数随之发散——格点规范理论中由有限体积屏蔽而非表达式奇异处理，量纲自洽。

代码-物理映射

映射原则： C1 的每个物理量在代码中均有直接载体，双向可查 (详见 C2-C8 映射表)： $-\nabla^2 \leftrightarrow \text{_Laplacian}$ ； $V \leftrightarrow \text{eigenvector/evecs}$ ； $\tau \leftrightarrow \text{VSV/tau}$ ； $\Phi \leftrightarrow \text{VPV/phi}$ ； $\gamma \leftrightarrow \text{gamma(n)}$ ； $e^{ip \cdot x} \leftrightarrow \text{MomentumPhase}$ 。

2.2 C2 拉普拉斯本征矢生成：多后端求解

核心算法

核心算法： 对固定时间片 t ，组装三维规范协变拉普拉斯算子 (_Laplacian ，2.1.3 之 (1))，用迭代本征求解器 (scipy/cupyx 的 ARPACK，或 QUDA 的 Lanczos) 求最小本征值对应的 N_e 个本征矢；随后按首位分量相位统一归一化 (renorm phase)，消除跨组态/跨时间片本征矢的任意 $U(1)$ 相位。

关键实现： 算子组装用 `opt_einsum` 一次收缩 + 移位 (`backend.roll`)，对全部格点向量化，无逐点循环：

```

1 def _Laplacian(F, U, U_dag, latt_size):
2     backend = get_backend()
3     Lx, Ly, Lz, Lt = latt_size
4     F = F.reshape(Lz, Ly, Lx, Nc, -1)
5     return (
6         # - for SA with evals , + for LA with (12 - evals)
7         6 * F
8         - (
9             contract("zyxab,zyxbc->zyxac", U[0], backend.roll(F, -1, 2))
10            + contract("zyxab,zyxbc->zyxac", U[1], backend.roll(F, -1, 1))
11            + contract("zyxab,zyxbc->zyxac", U[2], backend.roll(F, -1, 0))
12            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[0], F), 1, 2)
13            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[1], F), 1, 1)
14            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[2], F), 1, 0)
15        )
16    ).reshape(Lz * Ly * Lx * Nc, -1)

```

三种求解路径 (lattice/generator/eigenvector.py:234-312, 356-370)：

- `laplacian_cupy_numpy`: 纯 Python 组装 + `scipy/cupyx.sparse.linalg.eigsh` (SA 端、tol 控制), 最灵活, 可用于 numpy 后端;
- `laplacian_quda`: 把单时间片配置装入 QUDA 的 `QUDA_LAPLACE_DSLASH`, 用 `QUDA_EIG_TR_LANCZOS` 求 N_e 个本征对 (可开 Chebyshev 多项式加速), 返回前按 $2(N_d - 1)$ 重标定 (`eigenvector.py:307`), 与 (1) 的 [0,12] 量纲一致;
- `laplacian_quda_scipy`: 仅用 QUDA 提供拉普拉斯矩阵向量积, 外部仍走 ARPACK。

相位归一化 (renorm phase): 本征矢 $v^{(k)}$ 定义到整体相位 $e^{i\theta_k}$ 为止; 蒸馏法要求跨组态一致 (否则 perambulator/elemental 出现随机相位差)。代码取 `renorm_phase = angle(evecs[:, 0])` (`eigenvector.py:255`) 并对全体乘 $e^{-i\theta_k}$, 把每个本征矢的第一分量归为实数。这是格点蒸馏法工程实现中必要且非平凡的细节。

代码-物理映射

代码-物理映射:

代码符号	物理对象	物理含义与参考源
<code>U[mu]</code>	规范链接 $U_\mu(x)$	色旋转矩阵, <code>roll</code> 实现 $\hat{\mu}$ 方向平移; <code>lattice/generator/eigenvector.py:19-24</code>
<code>_Laplacian</code>	$-\nabla^2$ 算子	协变二点导数与对角项 $6F$ 之差; <code>lattice/generator/eigenvector.py:11-26</code>
<code>evals/evecs</code>	$(\lambda_k, v^{(k)})$	SA 端最小本征对; <code>lattice/generator/eigenvector.py:246-257</code>
<code>renorm_phase</code>	本征矢 $U(1)$ 相位约定	首分量归实, 跨组态一致; <code>lattice/generator/eigenvector.py:254-256</code>
<code>project_SU3</code>	$SU(3)$ 群流形投影	迭代 $U \mapsto \frac{1}{2}(U + U^{\dagger-1})$ 消漂移; <code>lattice/generator/eigenvector.py:55-65</code>
<code>stout_smear</code>	平滑规范场 $U^{(n)}$	供蒸馏平滑源使用 (见 C3) ; <code>lattice/generator/eigenvector.py:215-232</code>

复杂度: 组装 $O(VN_c^2)$ (6 次收缩); 迭代求解 $O(N_{\text{iter}} \cdot VN_c^2)$, ARPACK 每步一次算子作用; Lanczos 收敛迭代数远小于 $3V$ 维全对角化 $O((3V)^3)$, 为蒸馏法”只取低模”策略的效率基础 (推断, 未做基准测试)。

2.3 C3 Stout 平滑: $SU(3)$ 解析矩阵指数化

这是本仓库最精巧的数值物理模块。平滑规范场由 Morningstar-Peardon 三步构造 (<https://arxiv.org/abs/hep-lat/0311018>, 代码注释引用见 `lattice/generator/eigenvector.py:123-190` 与 `elemental` 同构实现)。

2.3.1 公式推导链

第一步 (定义) : 2 环 staple 求和与 Q 矩阵。对方向 μ 定义

$$C_\mu(x) = \rho \sum_{\nu \neq \mu} \left[U_\nu(x) U_\mu(x + \hat{\nu}) U_\nu^\dagger(x + \hat{\mu}) + U_\nu^\dagger(x - \hat{\nu} + \hat{\mu}) U_\mu(x - \hat{\nu}) U_\nu(x - \hat{\nu}) \right], \quad (7)$$

对应两条二环路 (plaquette 的”上一右一回”与”回一上一右”)。代码 `_stout_smear_ndarray:132-138` 用 `roll + @` 逐项实现。随后构造

$$Q_\mu \equiv \frac{i}{2} (C_\mu U_\mu^\dagger - U_\mu C_\mu^\dagger) - \frac{i}{6} \text{Tr} (C_\mu U_\mu^\dagger - U_\mu C_\mu^\dagger) \mathbb{K}, \quad (8)$$

即 $Q = \frac{i}{2} (\Omega^\dagger - \Omega)$ 的反对称部分取无迹投影 ($\Omega \equiv C U^\dagger$)。性质 (确证, 可直接验证): $Q^\dagger = Q$ (厄米)、 $\text{Tr } Q = 0$ 。

第二步 (公式) : 解析指数化。平滑链接定义为

$$U'_\mu(x) = e^{iQ_\mu(x)} U_\mu(x), \quad e^{iQ} \in SU(3). \quad (9)$$

对无迹厄米 3×3 矩阵 Q , 由 Cayley–Hamilton 定理 (Q 满足三次特征多项式), 指数函数可写为 Q 的二次多项式:

$$e^{iQ} = f_0 \mathbb{K} + f_1 Q + f_2 Q^2, \quad (10)$$

其中系数由 Q 的谱决定。设 Q 本征值为 $i\lambda_1, i\lambda_2, i\lambda_3$ ($\sum \lambda_k = 0$, 由无迹性), 令其参数化为 $\lambda_1 = u, \lambda_{2,3} = -\frac{u}{2} \pm iw$ 的插值形式; 对三个插值节点应用拉格朗日插值并整理 (推导要点: $f_0 = \sum_k e^{i\lambda_k} \lambda_j \lambda_l / [(\lambda_k - \lambda_j)(\lambda_k - \lambda_l)]$ 等三式对称化), 得到代码中的闭式表达 (`eigenvector.py:100-120`, 与 Morningstar–Peardon 附录一致):

$$u = \sqrt{c_1/3} \cos(\theta/3), \quad w = \sqrt{c_1} \sin(\theta/3), \quad (11)$$

$$c_1 \equiv \frac{1}{2} \text{Re Tr } Q^2, \quad c_0 \equiv \frac{1}{3} \text{Re Tr } Q^3, \quad \theta \equiv \arccos(c_0/c_0^{\max}), \quad c_0^{\max} = 2(c_1/3)^{3/2}, \quad (12)$$

$$f_0 = \frac{(u^2 - w^2)e^{2iu} + e^{iu}(8u^2 \cos w + 2iu(3u^2 + w^2) \text{sinc } w)}{9u^2 - w^2}, \quad (13)$$

$$f_1 = \frac{2ue^{2iu} - e^{iu}(2u \cos w - i(3u^2 - w^2) \text{sinc } w)}{9u^2 - w^2}, \quad (14)$$

$$f_2 = \frac{e^{2iu} - e^{iu}(\cos w + 3iu \text{sinc } w)}{9u^2 - w^2}. \quad (15)$$

其中 $c_0 \geq 0$ 时 $\theta = \arccos(c_0/c_0^{\max})$, $c_0 < 0$ 时取共轭分支 (代码 `parity` 标记 `eigenvector.py:95-96, 113-115`) ——由 $\text{Tr } Q^3$ 的符号选择 $e^{\pm iQ}$ 本征值分支, 保持 $\det e^{iQ} = 1$ 。

第三步 (数值细节) : 小 w 级数展开。 $w \rightarrow 0$ 时 $\text{sinc } w = \sin w/w$ 计算出现 $0/0$ 退化; 代码用 $\sin w/w$ 的泰勒展开截断至 w^8 阶 (`eigenvector.py:105`):

$$\text{sinc } w \approx 1 - \frac{w^2}{6} \left(1 - \frac{w^2}{20} \left(1 - \frac{w^2}{42} \left(1 - \frac{w^2}{72} \right) \right) \right), \quad |w| \leq 0.05, \quad (16)$$

乘开即 $1 - w^2/6 + w^4/120 - w^6/5040 + w^8/362880$ (确证, 逐项验算一致); $|w| > 0.05$ 用精确 $\sin w/w$ 。

极限与退化校验:

- $\rho \rightarrow 0$: $C_\mu \rightarrow 0 \Rightarrow Q \rightarrow 0 \Rightarrow e^{iQ} \rightarrow \mathbb{K}$, $U' \rightarrow U$ (不平滑) ——合理 ((7) 线性依赖 ρ)。

- $w \rightarrow 0, u \rightarrow 0: Q \rightarrow 0$, (10) 的 f 系数 $f_0 \rightarrow 1, f_1 \rightarrow 0, f_2 \rightarrow 0$ （级数展开确保无奇点）， $U' \rightarrow U$ ——一致。
- 保群性：由构造 Q 厄米无迹 $\Rightarrow e^{iQ}$ 么正 $\det = 1$ ；数值实现无显式再投影（project_SU3 仅在外
部可选调用），依赖公式的精确性——这是解析法相对数值 expm 的优势之一。

四种实现（多后端策略）：

实现	适用后端	特点
<code>_stout_smear_ndarray_naive</code>	numpy/cupy	复指数直接求值，教学清晰； <code>eigenvector.py</code> : 67–121
<code>_stout_smear_ndarray</code>	numpy/cupy	实部/虚部分离的预展开（避免复数幂），即默认路 径； <code>eigenvector.py</code> :123–190
<code>_stout_smear_cuda_kernel</code>	cupy	自定义 CUDA kernel (<code>stout_smear.cu</code>)，网格按 格点并行， $O(V)$ 线程； <code>eigenvector.py</code> :192–201
<code>_stout_smear_quda</code>	QUDA	调用库级 <code>smearSTOUT</code> ； <code>eigenvector.py</code> :203–213

代码-物理映射

代码-物理映射：

代码符号	物理对象	物理含义与参考源
<code>Q</code>	厄米无迹生成元 iQ	(8) 之 Q ； <code>eigenvector.py</code> :88–90
<code>c0, c1, c0_max</code>	谱不变量 $c_0, c_1, c_0^{\max}$	(12)； <code>eigenvector.py</code> :91–96
<code>u, w</code>	本征值参数化 u, w	(11)； <code>eigenvector.py</code> :98–99
<code>f0, f1, f2</code>	插值系数 f_0, f_1, f_2	(10)–(15)； <code>eigenvector.py</code> :110–120
<code>sinc_w</code>	$\sin w/w$ 小参数近似	(16)； <code>eigenvector.py</code> :105–108
<code>rho</code>	平滑强度 ρ	无量纲，典型 0.1； <code>eigenvector.py</code> :215

复杂度：每平滑步每方向 $O(VN_c^3)$ （6 次 3×3 收缩 + 指数化）；CUDA kernel 以 $O(V/2)$ 线程块并行，是 numpy 版的逐点向量化（未测速，推断）。

2.4 C4 Elemental 生成：导数展开与动量投影

核心算法

核心算法：对每个本征矢对 (i, j) 计算 (4)：先按导数链 $\nabla^{(n)}$ 作用（协变差分，`_nD`），再乘动量
相位 $e^{ip \cdot x}$ 并在全空间求和。高阶导数链用”左/右求导分配”技巧（积分分部）拆分为多个基础
差分，自动枚举 2^n 种分配并加权。

协变导数实现 (`lattice/generator/elemental.py`:279–288)：


```

1  def _nD(self, V, U, deriv):
2      backend = get_backend()
3
4      for d in deriv:
5          Vf = backend.roll(V, -1, 3 - d)
6          UVf = contract("zyxab,ezyxb->ezyxa", U[d], Vf)
7          UdV = contract("zyxba,ezyxb->ezyxa", U[d].conj(), V)
8          UbdVb = backend.roll(UdV, 1, 3 - d)
9          V = UVf - UbdVb
10     return V

```

导数链的自动枚举与分部积分 (elemental.py:309-337) :

```

1  for pick_nabla in range(2 ** len(derivative)):
2      pick_right = []
3      pick_left = []
4      pick = pick_nabla
5      for direction in derivative:
6          if pick & 1:
7              pick_right.append(direction)
8          else:
9              pick_left.append(direction)
10         pick >>= 1
11     coeff = (-1) ** len(pick_right)
12     right = self._nD(V, U[:, t], pick_right)
13     left = self._nD(V, U[:, t], pick_left[::-1])
14     for momentum_idx, momentum in enumerate(self.momentum_list):
15         if self.stochastic_coeff is None:
16             VPV[derivative_idx, momentum_idx] += contract(
17                 "zyx,ezyxc,fzyxc->ef",
18                 coeff * momentum_phase.get(momentum),
19                 left.conj(),
20                 right,
21             )
22         else:
23             VPV[derivative_idx, momentum_idx] += contract(
24                 "zyx,ezyxc,fzyxc,ef->ef",
25                 coeff * momentum_phase.get(momentum),
26                 left.conj(),
27                 right,
28                 self.stochastic_coeff,
29             )

```

上段代码把 n 阶导数链的所有 2^n 种左/右分配逐一遍历 (pick_nabla 的二进制位)，对”右求导”贡献系数 $(-1)^{\#right}$ ——这正是分部积分 $\int v^\dagger \nabla v = -\int (\nabla v)^\dagger v$ 在格点上的离散实现，同时利用 $(\nabla v)^\dagger = v^\dagger \overleftarrow{\nabla}$ 的厄米性把全部求导集中在右边执行，复用同一 _nD 内核。

动量投影: MomentumPhase (lattice/insertion/phase.py:41-46) 把 $e^{i\mathbf{p}\cdot\mathbf{x}} = \prod_\mu e^{2\pi i n_\mu x_\mu / L_\mu}$ 分解为三向一维相位数组的乘积并缓存，避免逐点复数幂。

随机混合 (blending): 当 is_blending=True 时，对”全本征矢空间 N_e 分为若干块 (totNe 组) 但只取其中 usedNe 子集”的随机稀释方案，构造修正系数矩阵 stochastic_coeff (elemental.py:61-95)：块内对角元 N_t/u 、非对角元 $N_t(N_t - 1)/[u(u - 1)]$ 、跨块元 $N_t N_s / (uv)$ (u, v 为块内实际使用数)，乘入收缩以无偏估计全空间结果——对应蒸馏法中”部分本征矢 + 随机噪声”的混合估计 (与 C13 噪声

矢量生成器配套)。

代码-物理映射

代码-物理映射:

代码符号	物理对象	物理含义与参考源
derivative(n)	导数链 $\nabla^{(n)}$ 编码	三进制方向码 (0=dx,1=dy,2=dz) ; lattice/ insertion/derivative.py:23-33
_nD	协变差分作用 $V \mapsto \nabla^{(n)}V$	(4) 之 $\nabla^{(n)}v$; elemental.py:279-288
num_derivative	导数组合数 N_{deriv}	(5); elemental.py:48
MomentumPhase	平面波 $e^{i\mathbf{p}\cdot\mathbf{x}}$	方向分解 + 缓存; insertion/phase.py:6-53
VPV	顶点矩阵 Φ	$[N_{\text{deriv}}, N_{\text{mom}}, N_e, N_e]$; elemental.py:56
stochastic_coeff	混合无偏权重	稀释修正系数; elemental.py:71-95

复杂度: 每时间片每导数每动量一次 $O(VN_e^2N_c)$ 收缩, 总量 $O(N_tN_{\text{deriv}}N_{\text{mom}}VN_e^2N_c)$; _nD 每次 $O(VN_eN_c)$ 。与朴素逐点实现相比, 全部算子作用走 roll+contract 批量化, 无 Python 层逐格点循环 (推断的复杂度上界, 未实测)。

2.5 C5 Perambulator 生成: Dirac 求逆与 $V^\dagger SV$

核心算法

核心算法: 对源时间片 t_i 与每个本征矢 e , 把 $V^\dagger$ 的第 e 列作为点源 (单旋量分量), 以 QUDA 求解 $SV_e = M^{-1}V_e$ (每源 $N_s = 4$ 次求逆, 或 MRHS 模式一次批量); 再收缩 $V^\dagger \cdot SV$ 得到 $\tau_e = V^\dagger SV_e$, 逐本征矢填充 (3)。

核心循环 (lattice/generator/perambulator.py:169-208) :

```

1   for eigen in range(Ne):
2       cp.cuda.runtime.deviceSynchronize()
3       s = perf_counter()
4       if self.MRHS:
5           print("Warning: use MRHS.")
6           V_MRHS = MultiLatticeFermion(latt_info, Ns)
7           for spin in range(Ns):
8               data = V_MRHS[spin].data.reshape(2, Lt, Lz, Ly, Lx // 2, Ns, Nc)
9               if gt * Lt <= t and (gt + 1) * Lt > t:
10                  data[:, t % Lt, :, :, spin, :] = backend.asarray(
11                      eigenvector_dagger[eigen, :, t % Lt, :, :, :].conj()
12                  )
13             SV_MRHS = dirac.invertMultiSrc(V_MRHS)
14             for spin in range(Ns):
15                 SV.reshape(Vol, Ns, Ns, Nc)[:, :, spin, :] = SV_MRHS[spin].data.reshape(Vol, Ns, Nc)
16             else:
17                 for spin in range(Ns):

```

```

18     V = LatticeFermion(latt_info) # V.data is double prec.
19     data = V.data.reshape(2, Lt, Lz, Ly, Lx // 2, Ns, Nc)
20     if gt * Lt <= t and (gt + 1) * Lt > t:
21         data[:, t % Lt, :, :, :, spin, :] = backend.asarray(
22             eigenvector_dagger[eigen, :, t % Lt, :, :, :, :].conj()
23         ) # [Ne, etzyx, Nc]
24     SV.reshape(Vol, Ns, Ns, Nc)[:, :, spin, :] = dirac.invert(V).data.reshape(Vol, Ns, Nc) # .get()
25     cp.cuda.runtime.deviceSynchronize()
26     invert_time = perf_counter() - s
27
28     cp.cuda.runtime.deviceSynchronize()
29     s = perf_counter()
30     VSV[:, :, :, :, eigen] = contract(
31         "ketzyxa,etzyxija->tijk", backend.asarray(eigenvector_dagger), backend.asarray(SV), optimize=True
32     )

```

设计要点（确证，代码注释与结构）：

- $V^\dagger$ 放主机内存 (perambulator.py:117-128)：本征矢转置共轭后存 host 再按需转 device，省设备显存——大 N_e （如 70）时的显存关键优化；
- MRHS 批量 (perambulator.py:172-183)：一次构造 N_s 源的多源求逆 invertMultiSrc，减少求逆次数与同步开销（更耗显存，选项化）；
- 时间片独立：对每个 t_i 单独求逆，逐时间片推进，天然适配流式计算（配合 Dispatch 逐组态分发，见 C16）。

收缩公式 (perambulator.py:198-200)：

$$\tau_{tijk}^{(e)} = \sum_{x,c} V_{ke}^\dagger(x,t) [SV]_{ijc}(x,t), \quad \text{einsum: "ketzyxa,etzyxija->tijk"}, \quad (17)$$

其中 k 为本征矢指标、 i, j 为旋量指标、 c 为色指标；输出 τ 尺寸 $[N_t, N_s, N_s, N_e, N_e]$ 。

代码-物理映射

代码-物理映射:

代码符号	物理对象	物理含义与参考源
eigenvector_dagger	$V^\dagger$ ($N_e \times N_t \times V \times N_c$)	源侧本征矢; perambulator.py:117-128
dirac.invert	M^{-1} 求逆	QUDA Dirac 算子; perambulator.py:192
SV	$SV = M^{-1}V$ 中间量	求逆结果 ($N_t N_s N_c$); perambulator.py:101, 192
VSV	perambulator τ	(3), (17); perambulator.py:102, 198-200
mass, tol, maxiter	Dirac 参数	质量/求解容差/迭代上限; perambulator.py:56-58
MRHS	多源批量开关	显存换时间的选项; perambulator.py:42-44, 172-183

复杂度：每时间片 N_e 次（或 MRHS 下 N_e/N_s 次）Dirac 求逆，每次 $O(N_{\text{iter}}VN_cN_s)$ ；收缩每次 $O(VN_eN_s^2N_c)$ 。总 $O(N_tN_eN_{\text{iter}}VN_cN_s)$ ——求逆占主导（推断）。

2.6 C6 自动 Wick 收缩：符号化-图论双轨

这是项目最具原创性的工程部分：用户仅以味结构（如 "ud"、"uds"）和动量/不可约表示声明强子，系统自动完成全部 Wick 收缩并输出可直接计算的收缩图。

2.6.1 符号化收缩（quark_contract.py）

算法原理：把每个强子展开为夸克/反夸克符号（Qurak，携带 flavor + tag + 时间片），按费米子场反对易的 Wick 定理递归配对：每次取第一个反夸克为源，枚举所有同味夸克为汇，插入传播子 $S^f(\text{sink}, \text{source})$ 并累计符号因子；递归直至耗尽（lattice/quark_contract.py:216-245）：

```

1 def _quark_contract(symbol_list, result_list, result, degenerate):
2     if symbol_list == []:
3         result_list.append(Mul(*result))
4         return
5     for i, src in enumerate(symbol_list):
6         if src.anti:
7             break
8     for j, snk in enumerate(symbol_list):
9         if not snk.anti and snk.flavor == src.flavor:
10            if i > j:
11                symbol_list.pop(i)
12                symbol_list.pop(j)
13                factor = S(-1) ** (i - j - 1)
14            else:
15                symbol_list.pop(j)
16                symbol_list.pop(i)
17                factor = S(-1) ** (j - i)
18            if degenerate and (snk.flavor == "u" or snk.flavor == "d"):
19                prop = Propagator("q", src.tag, snk.tag)
20            else:
21                prop = Propagator(src.flavor, src.tag, snk.tag)
22            result.append(factor * prop)
23            _quark_contract(symbol_list, result_list, result, degenerate)
24            result.pop()
25            if i > j:
26                symbol_list.insert(j, snk)
27                symbol_list.insert(i, src)
28            else:
29                symbol_list.insert(i, src)
30                symbol_list.insert(j, snk)

```

符号因子的物理来源（确证，可对照标准场论教材）：配对跨越 $\delta = |i - j| - 1$ 个中间算子时，每跨越一个产生 -1 （费米子场反交换），故

$$\begin{aligned}
 \text{符号} &= (-1)^{|i-j|-1} \text{ (反夸克在右, } i > j \text{ 分支)}, \\
 &= (-1)^{|i-j|} \text{ (反夸克在左分支, 见代码 quark_contract.py:228-232)},
 \end{aligned} \tag{18}$$

两分支一致（总幂次等于跨越算子数）。退化质量处理：`degenerate=True` 时 u/d 统一标记为 q (`quark_contract.py:233-236`)，自动折叠 $u-d$ 简并流形。

输出：每个收缩项记为传播子编号的邻接矩阵（强子为顶点、传播子为边权），打包为 `Diagram` 符号对象——物理表达式与收缩图完全同构。

2.6.2 图表示与自动 einsum 下标 (QuarkDiagram)

核心算法

核心算法：`QuarkDiagram.analyse` (`lattice/quark_diagram.py:23-79`) 对邻接矩阵做 BFS 连通分量分解，为每个传播子与顶点自动生成 `opt_einsum` 下标（旋量字符集 `mnop...` 与色字符集 `abcdef...`），拼接成 `subscripts` 字符串——把“画图”直接变成“一次 einsum 调用”。

```

1  def analyse(self) -> None:
2      from copy import deepcopy
3
4      adjacency_matrix = deepcopy(self.adjacency_matrix)
5      num_vertex = len(adjacency_matrix)
6      visited = [False] * num_vertex
7      for idx in range(num_vertex):
8          if visited[idx]:
9              continue
10         propagators = []
11         visited[idx] = True
12         queue = [idx]
13         while queue != []:
14             i = queue.pop(0)
15             for j in range(num_vertex):
16                 path = adjacency_matrix[i][j]
17                 if path != 0:
18                     adjacency_matrix[i][j] = 0
19                     if not visited[j]:
20                         visited[j] = True
21                         queue.append(j)
22                     if isinstance(path, int):
23                         propagators.append([path, i, j])
24                     elif isinstance(path, list):
25                         for _path in path:
26                             propagators.append([_path, i, j])
27                 else:
28                     raise ValueError(f"Invalid value {path} in the adjacency_matrix")
29         if propagators == []:
30             continue
31         vertex_operands = []
32         vertex_subscripts = []
33         propagator_operands = []
34         propagator_subscripts = []
35         node = 0
36         for propagator in propagators:
37             propagator_operands.append(propagator)
38             propagator_subscripts.append(_SUB_M[node + 1] + _SUB_A[node + 1] + _SUB_M[node] + _SUB_A[node])
39             if propagator[1] not in vertex_operands:

```

```

40         vertex_operands.append(propagator[1])
41         vertex_subscripts.append(_SUB_M[node] + _SUB_A[node])
42     else:
43         i = vertex_operands.index(propagator[1])
44         vertex_subscripts[i] = _SUB_M[node] + _SUB_A[node] + vertex_subscripts[i]
45     if propagator[2] not in vertex_operands:
46         vertex_operands.append(propagator[2])
47         vertex_subscripts.append(_SUB_M[node + 1] + _SUB_A[node + 1])
48     else:
49         i = vertex_operands.index(propagator[2])
50         vertex_subscripts[i] = vertex_subscripts[i] + _SUB_M[node + 1] + _SUB_A[node + 1]
51     node += 2
52     for key in range(len(propagator_subscripts)):
53         propagator_subscripts[key] = propagator_subscripts[key][0::2] + propagator_subscripts[key][1::2]
54     for key in range(len(vertex_subscripts)):
55         vertex_subscripts[key] = vertex_subscripts[key][0::2] + vertex_subscripts[key][1::2]
56     self.operands.append([propagator_operands, vertex_operands])
57     self.subscripts.append(",".join(propagator_subscripts) + "," + ",".join(vertex_subscripts))

```

多时间片收缩 (quark_diagram.py:233-258): 对传播子/顶点按时间片数组取值, 非整数时间片自动加 t 轴前缀并输出 $\rightarrow t$, 一次批处理全部时间片 (compute_diagrams_multitime), 避免逐 t Python 循环。

2.6.3 图化简 (Diagram.simplify)

核心算法

核心算法: Diagram.simplify (quark_diagram.py:330-553): BFS 求连通分量, 将大图拆为独立子图乘积; 顶点按 (时间, 类型) 排序, 同类型顶点遍历全部排列并取 SHA-256 哈希最小者作为规范形式——实现”同一物理图唯一表示”, 使重复子图可缓存复用。

核心效果 (确证, tests/test_simplify.py 六例覆盖):

- 冗余顶点剔除、连通分量拆分 (乘积化)、同构图归一 (排列哈希最小化);
- 传播子编号按序重排, 确保哈希稳定跨运行;
- 化简失败时回退原图并告警 (quark_diagram.py:651-657), 不中断计算。

代码-物理映射

代码-物理映射:

代码符号	物理对象	物理含义与参考源
Qurak(flavor,tag,anti)	夸克/反夸克场 $\psi_f, \bar{\psi}_f$	非对易符号; quark_contract.py:17-36
Propagator(flavor,src,snk)	传播子 $S^f(y, x)$	Wick 收缩结果; quark_contract.py:39-61

代码符号	物理对象	物理含义与参考源
HadronFlavorStructure	强子味结构	介子/重子/反重子三型; quark_contract.py:64-107
adjacency_matrix	收缩图 (Wick 图)	顶点 = 强子, 边 = 传播子编号; quark_diagram.py:15-21
subscripts	einsum 收缩模式	旋量/色指标自动分配; quark_diagram.py:60-79
Diagram.simplify	图规范形	连通拆分 + 哈希最小化; quark_diagram.py:330-553

正确性与复杂度：收缩结果与标准手写 Wick 收缩逐项比对 (tests/test_quark_contract.py, meson-meson / baryon-antibaryon / 退化 u-d 三例确证); n 对夸克的全收缩数为双阶乘 $(2n-1)!!$, 符号化枚举是指数级但物理上 $n \leq 6$ (强子数 ≤ 6), 可接受; 图化简的哈希归一使大计算 (多源多算子矩阵元) 中重复图只算一次 (推断收益, 未实测)。

2.7 C7 群论不可约表示框架： O_h^D 小群约化

2.7.1 物理对象

物理图像

物理图像：单强子插值场按 O_h^D (立方双盖群, 96 个元素: 24 旋转 $\times$ 2 (2π 旋转) $\times$ 2 (宇称)) 的不可约表示变换; 运动强子按动量 $\mathbf{p}$ 的小群 (stabilizer) 不可约表示分类。双粒子系统取总动量静止系后按 O_h^D 不可约表示 $\Lambda = A_1, A_2, E, T_1, T_2, G_1, G_2, H$ 分类, 投影算子将裸算子组合投影到指定 Λ ——这是两粒子谱分析的群论基础 (参考: <https://arxiv.org/abs/1607.06738> eq.(4.11), 代码注释 lattice/symmetry/two_particle.py:4-6)。

2.7.2 硬编码不可约表示库

hardcoded_rep.py (11,613 行) 预生成了 O_h^D 全部不可约表示矩阵 (含 O_h^D 自身 8 个不可约表示与全部小群 Dic_4, Dic_2, Dic_3, C_4 的表示) 以及群乘法表 (hardcoded_rep.py:104-120 的 OhD_Multiply_table)。数据由 utils.generate_hardcoded_code (lattice/symmetry/utils.py:30-46) 等脚本生成, 运行期零构造开销——这是“以生成期算力换运行期确定性”的典型工程决策。

小群约化 (lattice/symmetry/gen_hardcoded_rep.py:226-245): 对动量 $\mathbf{p}$ 的 (小群不可约表示 ρ , O_h^D 不可约表示 D), 约化矩阵为

$$R_{i\alpha}^{(p)} = \text{norm} \left[\sum_{g \in O_h^D} D_{ij}(g) \rho(g) \right], \quad \text{einsum: "ij,kk->kji"} \text{ 遍历全部群元}, \quad (19)$$

其中 $\rho(g)$ 经 Wigner 旋转回参考动量系的对应群元:

$$W(g, \mathbf{p}) = R_f^{-1} g R_i, \quad R_i \mathbf{p}_{\text{ref}} = \mathbf{p}, \quad (20)$$

R_i, R_f 由 `refRotateDict` (`group_generator.py:288-364`) 按动量查表。`HadronIrrepRow.transform` (`lattice/hadron_irrep.py:132-140`) 用 (20) 将强子插值场在任意旋转下变换, 进而实现 `expr_little_group_projection` (`hadron_irrep.py:178-195`) 的群平均投影。

正确性论证: 群元素由生成元 c_{4y}, c_{4z} 的积穷举生成并验证乘法表闭环 (`genMatrixGroup0hD, gen_hardcoded_rep.py:14-64`); 小群约化矩阵与”共线数组归一化” (`utils.check_and_normalize_arrays`) 逐列校验, 非共线即报错——保证约化矩阵的线性正确性。

代码-物理映射

代码-物理映射:

代码符号	物理对象	物理含义与参考源
<code>OhD_irreps/Dic*_irreps</code> <code>OhD_Multiply_table</code>	O_h^D 及各小群不可约表示 群乘法表	硬编码矩阵库; <code>hardcoded_rep.py</code> 96 元闭环验证; <code>hardcoded_rep.py</code> : 104-120
<code>genLittleGroupIrrep</code>	小群不可约表示 ρ_p	按 $ \mathbf{p} ^2$ 选小群; <code>gen_hardcoded_rep.py</code> : 162-223
<code>wignerRotate</code> <code>reductionToLittleGroup</code>	Wigner 旋转 $W(g, \mathbf{p})$ 约化矩阵 R	(20); <code>gen_hardcoded_rep.py:145-159</code> (19); <code>gen_hardcoded_rep.py:226-245</code>
<code>HadronIrrepRow.transform</code>	插值场变换	群元对强子场的作用; <code>hadron_irrep.py</code> : 132-140
<code>Fermion_generator</code>	自旋 1/2 表示生成元	c_{4y}, c_{4z} 显式矩阵; <code>group_generator.py</code> : 5-23

复杂度: 运行期约化一次 $O(96 \times d_\Lambda d_{\Lambda'})$ 次矩阵运算, 群构造与校验全部离线完成——相对”运行期动态生成群表”(需乘法表构建 $O(96^2)$ 次矩阵乘) 为显著常数优化 (推断)。

2.8 C8 两点关联函数收缩

核心算法

核心算法: `twopoint` (`lattice/correlator/one_particle.py:12-77`) 对每个源时间片 t_{src} 组装 $\tau(t_f, t_i)$ 与 $\tau_{\text{bw}} = \gamma_5 \tau^\dagger \gamma_5$, 对每个算子组合执行一次六路张量收缩, 直接给出 $C(t)$ ——即 (6) 的逐符号实现。

```

1  for it, t_src in enumerate(timeslices):
2      tau = perambulator[t_src, :, :, :, :usedNe, :usedNe]
3      # tau = backend.roll(perambulator[t, :, :, :, :usedNe, :usedNe], -t, 0)
4      if perambulator_bw is None:
5          tau_bw = contract("ii,tjiba,jj->tijab", gamma(15), tau.conj(), gamma(15))
6      else:
7          tmp = perambulator_bw[t_src, :, :, :, :usedNe, :usedNe]

```

```

8     tau_bw = contract("ii,tjiba,jj->tijab", gamma(15), tmp.conj(), gamma(15))
9     for idx in range(Nop):
10        phi = phis[idx]
11        gamma_src = contract("ij,xkj,kl->xil", gamma(8), phi[0].conj(), gamma(8))
12        ret[idx, it] = contract(
13            "tijab,xjk,xtbc,tklcd,yli,yad->t",
14            tau_bw,
15            phi[0],
16            backend.roll(phi[1], -t_src, 1),
17            tau,
18            gamma_src,
19            phi[1][:, t_src].conj(),
20        )

```

einsum 模式逐符号解释 (one_particle.py:64-72) :

$$C_{it} = \underbrace{\text{Tr}}_{tijkabcd} \left[\bar{\tau}_{tijab} \Phi_0^{xjk} \Phi_1^{xtbc} \tau_{tklcd} \gamma_{src}^{yli} \Phi_1^{\dagger yad} \right] \longleftrightarrow \text{"tijab,xjk,xtbc,tklcd,yli,yad->t"}, \quad (21)$$

其中 $\bar{\tau} = \tau_{bw}$ (C1 之 (6)), Φ_0 为源侧顶点 (含 $\gamma_4 \Phi^\dagger \gamma_4$ 共轭, 见 one_particle.py:63 的 gamma_src), Φ_1 为汇侧顶点并按 t_{src} 滚动平移 (时间平移不变性的显式利用, one_particle.py:68)。最终按源求和取平均并乘 -1 (费米子圈符号, one_particle.py:74-77), 与 (6) 的负号一致。

变体族 (同构核心, 复用同一 einsum 骨架) :

- twopoint_matrix: 算子 $\times$ 算子矩阵元 (关联矩阵);
- twopoint_profile / twopoint_indice: 保留本征矢指标 (a, b 维) 的轮廓函数, 用于谱密度/特征谱分析;
- twopoint_isoscalar: 连通 + 断连 (双圈) 贡献, N_f 因子折叠 (味求和), one_particle.py: 223-273;
- twopoint_matrix_multi_mom: 多动量批处理 (one_particle.py:326-413)。

复杂度: 单算子组合单源一次收缩 $O(N_e^2 N_s^2 N_c^2)$ (无空间维), 总量 $O(N_{op}^2 N_t N_e^2 N_s^2 N_c^2)$ ——空间维 V 在 elemental/perambulator 阶段已缩并, 这是蒸馏法相对点源法的核心性能优势 (推断, 量级对比见第 3 节)。

3 独特性与竞争力评估

4 关键片段与公式

核心公式集中于第 2 节 ((1)-(6)、(10)-(16)、(19)-(20)、(21)), 此处补充两个代表性代码片段: (1) Stout 平滑的 Q 组装与谱不变量计算 (与 (8)、(12) 对应) :

```

1     Q = rho * Q @ U_dag
2     Q = 0.5j * (Q.transpose(0, 1, 2, 3, 4, 6, 5).conj() - Q)
3     contract("...aa->...a", Q)[: ] -= 1 / Nc * contract("...aa->...", Q)[..., None]
4     Q_sq = Q @ Q

```

独特点	对比基准	优势与证据
解析 SU(3) 指数化	数值 <code>expm</code> (scipy) 或迭代级数	闭式 f_0, f_1, f_2 ((13)–(15)) 无迭代误差、可整体向量化、CUDA 友好；小 w 级数 (16) 消除奇点；四实现共享同一公式
拉普拉斯算子批量组装	逐格点循环	<code>roll+einsum</code> 单次收缩；三种求解路径（纯 Python/QUDA/混合）可切换
自动 Wick 收缩	手写收缩代码	符号化生成 + 图规范形哈希归一，杜绝人工转录错误；测试套件逐项比对 (<code>tests/test_quark_contract.py</code>)
自动 <code>einsum</code> 下标	手工编写收缩模式	BFS 自动分配旋量/色指标，图即表达式，表达式即 <code>einsum</code>
硬编码群论数据	运行期动态构造群表	11,613 行预生成不可约表示库，运行期零构造；乘法表闭环校验保证正确性
本征矢相位归一化	忽略相位约定的实现	<code>renorm_phase</code> 保证跨组态一致性，蒸馏法数值前提
$V^\dagger$ 主机驻留 + MRHS	全设备驻留 + 逐源求逆	显存/时间显式权衡，适配大 N_e (70 级)

表 9: 独特性评估（数据来源: 代码结构分析 + 单元测试，未做跨实现基准测试——标注为推断处除外）

```

5      c0 = contract("...aa->...", Q @ Q_sq).real / 3
6      c1 = contract("...aa->...", Q_sq).real / 2
7      c0_max = 2 * (c1 / 3) ** (3 / 2)
8      parity = c0 < 0
9      c0 = backend.abs(c0)
10     theta = backend.arccos(c0 / c0_max)
11     u = (c1 / 3) ** 0.5 * backend.cos(theta / 3)
12     w = c1**0.5 * backend.sin(theta / 3)
13     u_sq = u**2
14     w_sq = w**2
15     e_iu_real = backend.cos(u)
16     e_iu_imag = backend.sin(u)
17     e_2iu_real = backend.cos(2 * u)
18     e_2iu_imag = backend.sin(2 * u)
19     cos_w = backend.cos(w)
20     sinc_w = 1 - w_sq / 6 * (1 - w_sq / 20 * (1 - w_sq / 42 * (1 - w_sq / 72)))
21     large = backend.abs(w) > 0.05
22     w_large = w[large]
23     sinc_w[large] = backend.sin(w_large) / w_large
24     f_denom = 1 / (9 * u_sq - w_sq)

```

(2) 小群约化矩阵的群平均投影（与 (19) 对应）：

```

1  def reductionToLittleGroup(momentum, OhD_irrep_name, parity, little_group_irrep_name):
2      """
3      Reduce the OhD irrep to the little group irrep.
4      """
5      p = momentumSimplify(momentum)
6      matrix_little_group_irrep = genLittleGroupIrrep(p, little_group_irrep_name)
7      matrix_OhD_irrep = genIrrepOhD(OhD_irrep_name, parity)
8      ndim_little_group = matrix_little_group_irrep["iden"].shape[0]
9      ndim_OhD = matrix_OhD_irrep["iden"].shape[0]
10     reduction_matrix_colinear = np.full((ndim_little_group, ndim_OhD, ndim_OhD), S(0))
11     reduction_matrix = np.full((ndim_little_group, ndim_OhD), S(0))
12     for key in matrix_little_group_irrep.keys():
13         reduction_matrix_colinear += np.einsum(
14             "ij,kk->kji",
15             np.array(matrix_OhD_irrep[key]),
16             np.array(matrix_little_group_irrep[key]),
17         )
18     for i in range(ndim_little_group):
19         reduction_matrix[i] = check_and_normalize_arrays(reduction_matrix_colinear[i])
20     return reduction_matrix

```

5 参考源清单

参考源	类型	内容/作用
lattice/generator/eigenvector.py:11-370	仓库	拉普拉斯组装、stout 三实现、三种本征求解路径（C2/C3）
lattice/generator/perambulator.py:146-209	仓库	perambulator 求逆 + 收缩（C5）

参考源	类型	内容/作用
<code>lattice/generator/elemental.py:279-338</code>	仓库	<code>_nD</code> 协变导数与导数链枚举 (C4)
<code>lattice/quark_contract.py:110-245</code>	仓库	Wick 收缩符号化引擎 (C6)
<code>lattice/quark_diagram.py:15-553</code>	仓库	图表示、自动下标、化简 (C6)
<code>lattice/correlator/one_particle.py:12-413</code>	仓库	两点函数及其变体 (C8)
<code>lattice/symmetry/gen_hardcoded_rep.py:14-245</code>	仓库	群生成、Wigner 旋转、约化 (C7)
<code>lattice/symmetry/hardcoded_rep.py:1-11613</code>	仓库	硬编码不可约表示库 (C7)
<code>lattice/symmetry/group_generator.py:5-388</code>	仓库	生成元与参考动量表 (C7)
<code>tests/test_simplify.py:72-367</code>	仓库	图化简六例验证 (C6)
<code>tests/test_quark_contract.py:145-223</code>	仓库	Wick 收缩验证 (C6)
<code>example/gen_two_particle_corr_mom.py:48-168</code>	仓库	双粒子关联函数端到端示例 (C1/C8)
https://arxiv.org/abs/hep-lat/0311018	文献	Morningstar–Peardon: stout 平滑 (C3)
https://arxiv.org/abs/0905.2160	文献	Peardon 等: 蒸馏法原始文献 (C1)
https://arxiv.org/abs/1607.06738	文献	两粒子群论基 (eq.(4.11)) (C7, 代码注释所引)

6 结论

结论

穷尽剖析结论: 核心部分 17 项 (8 深挖 / 5 浅析 / 4 排除), 计数闭合。最强竞争力排序: (1) 解析 $SU(3)$ 指数化——蒸馏法工程中平滑算子的教科书级闭式实现, 四后端一致; (2) 符号化 Wick 收缩 + 图自动 `einsum`——把物理表达式到数值收缩的人工环节全部自动化, 图规范形保证可缓存; (3) 硬编码 O_h^D 群论库——以 11,613 行生成数据换取运行期确定性与零构造开销。三者共同支撑”蒸馏法全流程从物理声明到数值结果”的端到端自动化, 这是本仓库区别于通用线性代数库/单模块格点包的独特竞争力。

局限与风险

未验证/局限: (1) 复杂度结论均为代码结构推导 (标注”推断”), 仓库无基准测试数据可核验; (2) CUDA kernel (`stout_smear.cu`) 与 `numpy` 实现的逐符号一致性仅由同源公式保证, 未见差分测试; (3) 硬编码群论库的生成脚本 (`utils.generate_hardcoded_code`) 在库中无复现入口, 数据可信度依赖 ‘`hardcoded_rep.py`’ 头部的乘法表校验; (4) 双粒子基构造 (C10, 浅析) 仅实现 $J \leq 1$ 与 $p^2 \leq 9$ 的动量壳, 扩展需回归群论生成器; (5) 随机混合 (blending) 与噪声矢量 (C13) 的统计无偏性无方差分析文档。